

Python Refresher

Why choose Python

In this section, we will learn why Python is a popular and powerful choice for data science.

Variables, Data Types, and Typecasting

In this section, we will learn about variables, data types, and typecasting in Python to store and convert data effectively. ## Variables

- Containers for storing data values.
- No need to declare data type explicitly.

```
name = "Alice"  
age = 25  
is_student = True
```

Data Types

Type	Example	Description
int	10 , -5	Integer numbers
float	3.14 , -0.5	Decimal numbers
str	"hello"	Text (string)
bool	True , False	Boolean values
list	[1, 2, 3]	Ordered, mutable collection
tuple	(1, 2, 3)	Ordered, immutable collection

Type	Example	Description
dict	{"a": 1}	Key-value pairs

Typecasting (Type Conversion)

- Convert data from one type to another using built-in functions:

```
# str to int
x = int("10")      # 10

# int to str
y = str(25)        # "25"

# float to int
z = int(3.9)        # 3 (truncates, not rounds)

# list from string
lst = list("abc")   # ['a', 'b', 'c']
```

Quick Tips

- Use `type(variable)` to check a variable's data type.
- Typecasting errors can happen if the value isn't compatible:

```
int("hello") # ValueError
```

String and String Methods

In this section, we will learn about strings and string methods in Python to work with and manipulate text data.

What is a String?

- A string is a sequence of characters enclosed in single (`'`) or double (`"`) quotes.

```
name = "Alice"  
greeting = 'Hello'
```

Multiline Strings

- Use triple quotes (`'''` or `"""`) for multiline text.

```
message = """This is  
a multiline  
string."""
```

String Indexing and Slicing

- Indexing starts at 0.

```
text = "Python"  
text[0]    # 'P'  
text[-1]   # 'n' (last character)  
text[0:2]  # 'Py'  
text[:3]   # 'Pyt'  
text[3:]   # 'hon'
```

String Immutability

- Strings cannot be changed after creation.

```
text[0] = 'J' # Error
```

Common String Methods

Method	Description
<code>str.lower()</code>	Converts to lowercase
<code>str.upper()</code>	Converts to uppercase
<code>str.strip()</code>	Removes leading/trailing spaces
<code>str.replace(old, new)</code>	Replaces substring
<code>str.split(sep)</code>	Splits string into a list
<code>str.join(list)</code>	Joins list into string
<code>str.find(sub)</code>	Returns index of first occurrence
<code>str.count(sub)</code>	Counts occurrences of substring
<code>str.startswith(prefix)</code>	Checks if string starts with value
<code>str.endswith(suffix)</code>	Checks if string ends with value
<code>str.isdigit()</code>	Checks if all chars are digits
<code>str.isalpha()</code>	Checks if all chars are letters
<code>str.isalnum()</code>	Checks if all chars are letters/digits

Examples

```
"hello".upper()           # 'HELLO'
" Hello ".strip()         # 'Hello'
"hello world".split()     # ['hello', 'world']
"-".join(["2025", "04", "14"]) # '2025-04-14'
"python".find("th")       # 2
```

String Formatting (f-strings)

```
name = "Alice"
age = 30
f"Hello, {name}. You are {age} years old."
# 'Hello, Alice. You are 30 years old.'
```

Operators in Python

In this section, we will learn about different types of operators in Python and how they are used in expressions.

1. Arithmetic Operators Used for basic mathematical operations.

Operator	Description	Example (a=10, b=5)
+	Addition	a + b # 15
-	Subtraction	a - b # 5
*	Multiplication	a * b # 50
/	Division	a / b # 2.0
//	Floor Division	a // b # 2
%	Modulus	a % b # 0
**	Exponentiation	a ** b # 100000

2. Comparison Operators

Compare values and return `True` or `False`.

Operator	Description	Example (a=10, b=5)
==	Equal to	a == b # False
!=	Not equal to	a != b # True

Operator	Description	Example (a=10, b=5)
>	Greater than	a > b # True
<	Less than	a < b # False
>=	Greater or equal	a >= b # True
<=	Less or equal	a <= b # False

3. Logical Operators

Used to combine conditional statements.

Operator	Description	Example (x=True, y=False)
and	Both True	x and y # False
or	Either True	x or y # True
not	Negation	not x # False

4. Bitwise Operators

Perform bit-level operations.

Operator	Description	Example (a=5, b=3)
&	AND	a & b # 1
\	OR	a \ b # 7

5. Assignment Operators

Used to assign values to variables.

Operator	Example (a=10)	Equivalent to
=	a = 5	a = 5

Operator	Example (a=10)	Equivalent to
<code>+=</code>	<code>a += 5</code>	<code>a = a + 5</code>
<code>-=</code>	<code>a -= 5</code>	<code>a = a - 5</code>
<code>*=</code>	<code>a *= 5</code>	<code>a = a * 5</code>
<code>/=</code>	<code>a /= 5</code>	<code>a = a / 5</code>
<code>//=</code>	<code>a //= 5</code>	<code>a = a // 5</code>
<code>%=</code>	<code>a %= 5</code>	<code>a = a % 5</code>
<code>**=</code>	<code>a **= 5</code>	<code>a = a ** 5</code>

6. Membership & Identity Operators

Check for presence and object identity.

Operator	Description	Example (lst=[1,2,3] , x=2)
<code>in</code>	Present in sequence	<code>x in lst # True</code>
<code>not in</code>	Not present	<code>x not in lst # False</code>
<code>is</code>	Same object	<code>a is b # False</code>
<code>is not</code>	Different object	<code>a is not b # True</code>

Taking input from the user

In this section, we will learn how to take input from the user in Python and use it in our programs. `## Basic Usage`

```
name = input("Enter your name: ")
print("Hello", name)
```

Note: `input()` always returns a string.

Type Conversion

You can always convert the string output of `input()` function to other supported data types

```
age = int(input("Enter your age: "))
price = float(input("Enter the price: "))
```

Operator Precedence

Python follows **PEMDAS** (Parentheses, Exponents, Multiplication/Division, Addition/Subtraction). The order of operations in Python is:

1. **Parentheses** `()` – Highest precedence, operations inside parentheses are evaluated first.
2. **Exponents** `**` – Power calculations (e.g., `2 ** 3` → 8).
3. **Multiplication** `*`, **Division** `/`, **Floor Division** `//`, **Modulus** `%` – Evaluated from left to right.
4. **Addition** `+`, **Subtraction** `-` – Evaluated from left to right.

Example:

```
result = 10 + 2 * 3 # Multiplication happens first: 10 + (2 * 3)
print(result)

result = (10 + 2) * 3 # Parentheses first: (10 + 2) * 3 = 36
print(result)

result = 2 ** 3 ** 2 # Right-to-left exponentiation: 2 ** (3 ** 2)
print(result)
```

If else statements

In Python, conditional statements (`if`, `elif`, and `else`) are used to control the flow of a program based on conditions. These are essential in data science for

handling different scenarios in data processing, decision-making, and logic execution.

Basic `if` Statement

The `if` statement allows you to execute a block of code only if a condition is `True`.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

Explanation:

- The condition `x > 5` is checked.
- If `True`, the indented block under `if` runs.
- If `False`, nothing happens.

`if-else` Statement

The `else` block executes when the `if` condition is `False`.

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

Explanation:

- If `x > 5`, it prints the first message.
- Otherwise, the `else` block executes.

if-elif-else Statement

When multiple conditions need to be checked sequentially, use `elif` (short for "else if").

```
x = 5
if x > 10:
    print("x is greater than 10")
elif x > 5:
    print("x is greater than 5 but not more than 10")
elif x == 5:
    print("x is exactly 5")
else:
    print("x is less than 5")
```

Explanation:

- The conditions are checked from top to bottom.
- The first `True` condition executes, and the rest are skipped.

Using `if-else` in Data Science

Conditional statements are widely used in data science for filtering, cleaning, and decision-making.

Example: Categorizing Data

```
age = 25
if age < 18:
    category = "Minor"
elif age < 65:
    category = "Adult"
else:
    category = "Senior Citizen"

print("Category:", category)
```

Example: Applying Conditions on Pandas DataFrame

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Score': [85, 40, 75]}
df = pd.DataFrame(data)

df['Result'] = df['Score'].apply(lambda x: 'Pass' if x >= 50 else 'Fail')
print(df)
```

Summary

- `if` : Executes if the condition is `True` .
- `if-else` : Adds an alternative block if the condition is `False` .
- `if-elif-else` : Handles multiple conditions.
- Useful in data science for logic-based decision-making.

Match Case Statements

The `match-case` statement, introduced in Python 3.10, provides pattern matching similar to `switch` statements in other languages.

Syntax

```
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 400:
            return "Bad Request"
        case 404:
            return "Not Found"
        case 500:
            return "Internal Server Error"
        case _:
            return "Unknown Status"
```

```
return "Unknown Status"
```

```
print(http_status(200)) # Output: OK  
print(http_status(404)) # Output: Not Found
```

Features:

- The `_` (underscore) acts as a default case.
- Patterns can include literals, variable bindings, and even structural patterns.

Example: Matching Data Structures

Lets try to match a tuple using Match-Case statements

```
point = (3, 4)  
  
match point:  
    case (0, 0):  
        print("Origin")  
    case (x, 0):  
        print(f"X-Axis at {x}")  
    case (0, y):  
        print(f"Y-Axis at {y}")  
    case (x, y):  
        print(f"Point at ({x}, {y})")
```

String Formatting and F-Strings

String is arguably the most used immutable data types in Python. Python provides multiple ways to format strings, including the `format()` method and f-strings (introduced in Python 3.6).

1. Using `format()`

```
name = "Alice"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
# Output: My name is Alice and I am 25 years old.
```

Positional and Keyword Arguments

```
print("{0} is learning {1}".format("Alice", "Python")) # Using positional arguments
print("{name} is learning {language}".format(name="Alice", language="Python")) # Using keyword arguments
```

2. Using f-Strings (Recommended)

F-strings provide a cleaner and more readable way to format strings.

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Alice and I am 25 years old.
```

Expressions Inside f-Strings

```
a = 5
b = 10
print(f"Sum of {a} and {b} is {a + b}")
# Output: Sum of 5 and 10 is 15
```

Formatting Numbers

```
pi = 3.14159
print(f"Pi rounded to 2 decimal places: {pi:.2f}")
# Output: Pi rounded to 2 decimal places: 3.14
```

Padding and Alignment

```
print(f"{'Python':<10}") # Left-align
print(f"{'Python':>10}") # Right-align
print(f"{'Python':^10}") # Center-align
```

:<10 → The < symbol means left-align the text within a total width of 10 characters.

F-strings are the most efficient and recommended way to format strings in modern Python!

Loops in Python

Python has two main loops: `for` and `while`.

1. For Loop

Used to iterate over sequences like lists, tuples, and strings.

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Using `range()`

```
for i in range(3):
    print(i) # Output: 0, 1, 2
```

2. While Loop

Runs as long as a condition is `True`.

```
count = 0
while count < 3:
```

```
print(count)
count += 1
```

Output:

```
0
1
2
```

3. Loop Control Statements

- `break` → Exits the loop.
- `continue` → Skips to the next iteration.
- `pass` → Does nothing (used as a placeholder).

```
for i in range(5):
    if i == 3:
        break # Stops the loop at 3
    print(i)
```

List and List Methods

A list in Python is an ordered, mutable collection of elements. It can contain elements of different types.

Creating a List:

```
# Empty list
my_list = []

# List with elements
numbers = [1, 2, 3, 4, 5]
```

```
# Mixed data types
mixed_list = [1, "Hello", 3.14, True]
```

Common List Methods

Method	Description	Example
<code>append(x)</code>	Adds an element <code>x</code> to the end of the list.	<code>my_list.append(10)</code>
<code>extend(iterable)</code>	Extends the list by appending all elements from an iterable.	<code>my_list.extend([6, 7, 8])</code>
<code>insert(index, x)</code>	Inserts <code>x</code> at the specified <code>index</code> .	<code>my_list.insert(2, "Python")</code>
<code>remove(x)</code>	Removes the first occurrence of <code>x</code> in the list.	<code>my_list.remove(3)</code>
<code>pop([index])</code>	Removes and returns the element at <code>index</code> (last element if <code>index</code> is not provided).	<code>my_list.pop(2)</code>
<code>index(x)</code>	Returns the index of the first occurrence of <code>x</code> .	<code>my_list.index(4)</code>
<code>count(x)</code>	Returns the number of times <code>x</code> appears in the list.	<code>my_list.count(2)</code>
<code>sort()</code>	Sorts the list in ascending order.	<code>my_list.sort()</code>
<code>reverse()</code>	Reverses the order of the list.	<code>my_list.reverse()</code>
<code>copy()</code>	Returns a shallow copy of the list.	<code>new_list = my_list.copy()</code>
<code>clear()</code>	Removes all elements from the list.	<code>my_list.clear()</code>

Example Usage:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")
print(fruits)  # ['apple', 'banana', 'cherry', 'orange']

fruits.sort()
print(fruits)  # ['apple', 'banana', 'cherry', 'orange']
```

Tuples and Tuple Methods

A **tuple** in Python is an ordered, immutable collection of elements. It is similar to a list, but once created, its elements cannot be modified.

Creating a Tuple:

```
# Empty tuple
empty_tuple = ()

# Tuple with elements
numbers = (1, 2, 3, 4, 5)

# Mixed data types
mixed_tuple = (1, "Hello", 3.14, True)

# Single element tuple (comma is necessary)
single_element = (42,)
```

Common Tuple Methods

Method	Description	Example
<code>count(x)</code>	Returns the number of times <code>x</code> appears in the tuple.	<code>my_tuple.count(2)</code>
<code>index(x)</code>		<code>my_tuple.index(3)</code>

Method	Description	Example
	Returns the index of the first occurrence of x .	

Tuple Characteristics

- **Immutable:** Once created, elements cannot be changed.
- **Faster than lists:** Accessing elements in a tuple is faster than in a list.
- **Can be used as dictionary keys:** Since tuples are immutable, they can be used as keys in dictionaries.

Accessing Tuple Elements

```
my_tuple = (10, 20, 30, 40)

# Indexing
print(my_tuple[1]) # 20

# Slicing
print(my_tuple[1:3]) # (20, 30)
```

Tuple Packing and Unpacking

```
# Packing
person = ("Alice", 25, "Engineer")

# Unpacking
name, age, profession = person
print(name) # Alice
print(age) # 25
```

When to Use Tuples?

- When you want an **unchangeable** collection of elements.
- When you need a **faster** alternative to lists.
- When storing **heterogeneous data** (e.g., database records, coordinates).

Set and Set Methods

A **set** in Python is an **unordered**, **mutable**, and **unique** collection of elements. It does not allow duplicate values.

Creating a Set:

```
# Empty set (must use set(), not {})  
empty_set = set()  
  
# Set with elements  
numbers = {1, 2, 3, 4, 5}  
  
# Mixed data types  
mixed_set = {1, "Hello", 3.14, True}  
  
# Creating a set from a list  
unique_numbers = set([1, 2, 2, 3, 4, 4, 5])  
print(unique_numbers) # {1, 2, 3, 4, 5}
```

Common Set Methods

Method	Description	Example
<code>add(x)</code>	Adds an element <code>x</code> to the set.	<code>my_set.add(10)</code>
<code>update(iterable)</code>	Adds multiple elements from an iterable.	<code>my_set.update([6, 7, 8])</code>
<code>remove(x)</code>	Removes <code>x</code> from the set (raises an error if not found).	<code>my_set.remove(3)</code>

Method	Description	Example
<code>discard(x)</code>	Removes <code>x</code> from the set (does not raise an error if not found).	<code>my_set.discard(3)</code>
<code>pop()</code>	Removes and returns a random element.	<code>my_set.pop()</code>
<code>clear()</code>	Removes all elements from the set.	<code>my_set.clear()</code>
<code>copy()</code>	Returns a shallow copy of the set.	<code>new_set = my_set.copy()</code>

Set Operations

Operation	Description	Example
<code>union(set2)</code>	Returns a new set with all unique elements from both sets.	<code>set1.union(set2)</code>
<code>intersection(set2)</code>	Returns a set with elements common to both sets.	<code>set1.intersection(set2)</code>
<code>difference(set2)</code>	Returns a set with elements in <code>set1</code> but not in <code>set2</code> .	<code>set1.difference(set2)</code>
<code>symmetric_difference(set2)</code>		<code>set1.symmetric_difference(set2)</code>

Operation	Description	Example
	Returns a set with elements in either set1 or set2 , but not both.	
<code>issubset(set2)</code>	Returns True if set1 is a subset of set2 .	<code>set1.issubset(set2)</code>
<code>issuperset(set2)</code>	Returns True if set1 is a superset of set2 .	<code>set1.issuperset(set2)</code>

Example Usage:

In Python, sets support intuitive operators for common operations like union (`|`), intersection (`&`), difference (`-`), and symmetric difference (`^`). These have equivalent method forms too, like `.union()` , `.intersection()` , etc. Here's a quick example:

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Union - combines all unique elements
print(set1 | set2)           # {1, 2, 3, 4, 5, 6}
print(set1.union(set2))     # same result

# Intersection - common elements
print(set1 & set2)           # {3, 4}
```

```

print(set1.intersection(set2))# same result

# Difference - in set1 but not in set2
print(set1 - set2)           # {1, 2}
print(set1.difference(set2)) # same result

# Symmetric Difference - in either set, but not both
print(set1 ^ set2)           # {1, 2, 5, 6}
print(set1.symmetric_difference(set2))# same result

```

Key Properties of Sets:

- **Unordered:** No indexing or slicing.
- **Unique Elements:** Duplicates are automatically removed.
- **Mutable:** You can add or remove elements.

Dictionary and Dictionary Methods

A **dictionary** in Python is an **unordered**, **mutable**, and **key-value** pair collection. It allows efficient data retrieval and modification. Dictionaries in Python are ordered as of Python 3.7

Creating a Dictionary:

```

# Empty dictionary
empty_dict = {}

# Dictionary with key-value pairs
student = {
    "name": "Alice",
    "age": 25,
    "grade": "A"
}

# Using dict() constructor
person = dict(name="John", age=30, city="New York")

```

Accessing Dictionary Elements

```
# Using keys
print(student["name"]) # Alice

# Using get() (avoids KeyError if key doesn't exist)
print(student.get("age")) # 25
print(student.get("height", "Not Found")) # Default value
```

Common Dictionary Methods

Method	Description	Example
<code>keys()</code>	Returns all keys in the dictionary.	<code>student.keys()</code>
<code>values()</code>	Returns all values in the dictionary.	<code>student.values()</code>
<code>items()</code>	Returns key-value pairs as tuples.	<code>student.items()</code>
<code>get(key, default)</code>	Returns value for <code>key</code> , or <code>default</code> if key not found.	<code>student.get("age", 0)</code>
<code>update(dict2)</code>	Merges <code>dict2</code> into the dictionary.	<code>student.update({"age": 26})</code>
<code>pop(key, default)</code>	Removes key and returns its value (or <code>default</code> if key not found).	<code>student.pop("grade")</code>
<code>popitem()</code>	Removes and returns the last inserted key-value pair.	<code>student.popitem()</code>

Method	Description	Example
<code>setdefault(key, default)</code>	Returns value for <code>key</code> , else sets it to <code>default</code> .	<code>student.setdefault("city", "Unknown")</code>
<code>clear()</code>	Removes all items from the dictionary.	<code>student.clear()</code>
<code>copy()</code>	Returns a shallow copy of the dictionary.	<code>new_dict = student.copy()</code>

Example Usage:

```
student = {"name": "Alice", "age": 25, "grade": "A"}

# Adding a new key-value pair
student["city"] = "New York"

# Updating an existing value
student["age"] = 26

# Removing an item
student.pop("grade")

# Iterating over a dictionary
for key, value in student.items():
    print(key, ":", value)

# Output:
# name : Alice
# age : 26
# city : New York
```

Dictionary Comprehension:

```
# Creating a dictionary using comprehension
squares = {x: x**2 for x in range(1, 6)}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Key Properties of Dictionaries:

- **Unordered** (Python 3.6+ maintains insertion order).
- **Keys must be unique and immutable** (e.g., strings, numbers, tuples).
- **Values can be mutable** and of any type.

File Handling in Python

File handling allows Python programs to **read, write, and manipulate files** stored on disk. Python provides built-in functions for working with files.

Opening a File

Python uses the `open()` function to open a file.

Syntax

```
file = open("filename", mode)
```

- **filename** → The name of the file to open.
- **mode** → Specifies how the file should be opened.

File Modes

Mode	Description
'r'	Read (default) – Opens file for reading, raises an error if file does not exist.

Mode	Description
'w'	Write – Opens file for writing, creates a new file if not found , and overwrites existing content .
'a'	Append – Opens file for writing, creates a new file if not found , and appends content instead of overwriting.
'x'	Create – Creates a new file, but fails if the file already exists .
'b'	Binary mode – Used with <code>rb</code> , <code>wb</code> , <code>ab</code> , etc., for working with non-text files (e.g., images, PDFs).
't'	Text mode (default) – Used for text files (e.g., <code>rt</code> , <code>wt</code>).

Reading Files

Using `read()` – Read Entire File

```
file = open("example.txt", "r")
content = file.read()
print(content)
file.close() # Always close the file after use
```

Using `readline()` – Read Line by Line

```
file = open("example.txt", "r")
line1 = file.readline() # Reads first line
print(line1)
file.close()
```

Using `readlines()` – Read All Lines as List

```
file = open("example.txt", "r")
lines = file.readlines() # Reads all lines into a list
print(lines)
file.close()
```

Writing to Files

Using `write()` – Overwrites Existing Content

```
file = open("example.txt", "w") # Opens file in write mode
file.write("Hello, World!") # Writes content
file.close()
```

Space ()

Tab (\t)

Newline (\n)

Carriage
return (\r)

Form feed (\f)

Using `writelines()` – Write Multiple Lines

```
lines = ["Hello\n", "Welcome to Python\n", "File Handling\n"]

file = open("example.txt", "w")
file.writelines(lines) # Writes multiple lines
file.close()
```

Appending to a File

The `a` (append) mode is used to add content to an existing file without erasing previous data.

```
file = open("example.txt", "a")
file.write("\nThis is an additional line.")
file.close()
```

Using `with` Statement (Best Practice)

Using `with open()` ensures the file is **automatically closed** after execution.

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content) # No need to manually close the file
```

Checking if a File Exists

Use the `os` module to check if a file exists before opening it.

```
import os

if os.path.exists("example.txt"):
    print("File exists!")
else:
    print("File not found!")
```

Deleting a File

Use the `os` module to delete a file.

```
import os

if os.path.exists("example.txt"):
    os.remove("example.txt")
    print("File deleted.")
else:
    print("File does not exist.")
```

Working with Binary Files

Binary files (`.jpg` , `.png` , `.pdf` , etc.) should be opened in **binary mode** (`'b'`).

Reading a Binary File

```
with open("image.jpg", "rb") as file:
    data = file.read()
    print(data) # Outputs binary content
```

Writing to a Binary File

```
with open("new_image.jpg", "wb") as file:
    file.write(data) # Writes binary content to a new file
```

Summary of File Operations

Operation	Description	Example
Open file	Open a file	<code>file = open("example.txt", "r")</code>
Read file	Read all content	<code>file.read()</code>
Read line	Read one line	<code>file.readline()</code>
Read lines	Read all lines into list	<code>file.readlines()</code>
Write file	Write content (overwrite)	<code>file.write("Hello")</code>
Append file	Add content to the end	<code>file.write("\nMore text")</code>
		<code>os.path.exists("file.txt")</code>

Operation	Description	Example
Check file existence	Check before opening/deleting	
Delete file	Remove a file	<code>os.remove("file.txt")</code>

JSON module in Python

JSON (JavaScript Object Notation) is a lightweight data format used for data exchange between servers and applications. It is widely used in **APIs, web applications, and configurations**.

Python provides the `json` module to work with JSON data. You can import the `json` module like this:

```
import json
```

Converting Python Objects to JSON (Serialization)

Serialization (also called **encoding or dumping**) is converting a Python object into a JSON-formatted string.

`json.dumps()` – Convert Python object to JSON string

```
import json

data = {"name": "Alice", "age": 25, "city": "New York"}

json_string = json.dumps(data)
print(json_string) # Output: {"name": "Alice", "age": 25, "city": "New York"}
print(type(json_string)) # <class 'str'>
```

`json.dump()` – Write JSON data to a file

```
with open("data.json", "w") as file:  
    json.dump(data, file)
```

Converting JSON to Python Objects (Deserialization)

Deserialization (also called **decoding** or **loading**) is converting JSON-formatted data into Python objects.

`json.loads()` – Convert JSON string to Python object

```
json_data = '{"name": "Alice", "age": 25, "city": "New York"}'  
  
python_obj = json.loads(json_data)  
print(python_obj) # Output: {'name': 'Alice', 'age': 25, 'city':  
print(type(python_obj)) # <class 'dict'>
```

`json.load()` – Read JSON data from a file

```
with open("data.json", "r") as file:  
    python_data = json.load(file)  
  
print(python_data) # Output: {'name': 'Alice', 'age': 25, 'city':
```

Formatting JSON Output

You can format JSON for better readability using **indentation**.

```
formatted_json = json.dumps(data, indent=4)  
print(formatted_json)
```

Output:

```
{
  "name": "Alice",
  "age": 25,
  "city": "New York"
}
```

Summary of Common JSON Methods

Method	Description	Example
<code>json.dumps(obj)</code>	Converts Python object to JSON string	<code>json.dumps(data)</code>
<code>json.dump(obj, file)</code>	Writes JSON to a file	<code>json.dump(data, file)</code>
<code>json.loads(json_string)</code>	Converts JSON string to Python object	<code>json.loads(json_data)</code>
<code>json.load(file)</code>	Reads JSON from a file	<code>json.load(file)</code>

Object Oriented Programming in Python

Object-Oriented Programming (OOP) is a **programming paradigm** that organizes code into objects that contain both **data (attributes)** and **behavior (methods)**.

Key Concepts of OOP

Concept	Description
Class	A blueprint for creating objects.

Concept	Description
Object	An instance of a class with specific data and behavior.
Attributes	Variables that store data for an object.
Methods	Functions inside a class that define object behavior.
Encapsulation	Restricting direct access to an object's data.
Inheritance	Creating a new class from an existing class.
Polymorphism	Using the same method name for different classes.

1. Defining a Class and Creating an Object

Creating a Class

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand # Attribute
        self.model = model # Attribute

    def display_info(self): # Method
        return f"{self.brand} {self.model}"

# Creating an Object (Instance)
car1 = Car("Toyota", "Camry")
print(car1.display_info()) # Output: Toyota Camry
```

2. Encapsulation (Data Hiding)

Encapsulation prevents direct modification of attributes and allows controlled access using **getter and setter methods**.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private Attribute

    def get_balance(self): # Getter
        return self.__balance

    def deposit(self, amount): # Setter
        if amount > 0:
            self.__balance += amount

# Using Encapsulation
account = BankAccount(1000)
account.deposit(500)
print(account.get_balance()) # Output: 1500

```

◇ Why use encapsulation?

It protects data by restricting direct modification.

3. Inheritance (Reusing Code)

Inheritance allows a class (child) to inherit attributes and methods from another class (parent).

Example of Single Inheritance

```

class Animal:
    def speak(self):
        return "Animal makes a sound"

class Dog(Animal): # Inheriting from Animal
    def speak(self):
        return "Bark"

dog = Dog()
print(dog.speak()) # Output: Bark

```

◇ Why use inheritance?

It promotes **code reusability** and maintains a cleaner code structure.

4. Multiple Inheritance

A class can inherit from multiple parent classes.

```
class A:
    def method_a(self):
        return "Method A"

class B:
    def method_b(self):
        return "Method B"

class C(A, B): # Multiple Inheritance
    pass

obj = C()
print(obj.method_a()) # Output: Method A
print(obj.method_b()) # Output: Method B
```

◇ Why use multiple inheritance?

It allows a class to inherit **features from multiple parent classes**.

5. Polymorphism (Same Method, Different Behavior)

Polymorphism allows different classes to use the **same method name**.

Method Overriding Example

```
class Bird:
    def fly(self):
        return "Birds can fly"
```

```

class Penguin(Bird):
    def fly(self):
        return "Penguins cannot fly"

bird = Bird()
penguin = Penguin()

print(bird.fly())      # Output: Birds can fly
print(penguin.fly())  # Output: Penguins cannot fly

```

◇ Why use polymorphism?

It provides **flexibility** by allowing different classes to define the same method differently.

6. Abstraction (Hiding Implementation Details)

Abstraction is used to define a method **without implementing** it in the base class. It is achieved using **abstract base classes** (ABC module).

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass # No implementation

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side # Implemented in child class

square = Square(4)
print(square.area()) # Output: 16

```

◇ Why use abstraction?

It enforces **consistent implementation** across child classes.

7. Magic Methods (Dunder Methods)

Magic methods allow objects to behave like **built-in types**.

Example: `__str__()` and `__len__()`

Have a look at the code below:

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self): # String representation
        return f"Book: {self.title}"

    def __len__(self): # Define behavior for len()
        return self.pages

book = Book("Python Basics", 300)
print(str(book)) # Output: Book: Python Basics
print(len(book)) # Output: 300
```

8. Class vs. Static Methods

Method Type	Description	Uses self ?	Uses cls ?
Instance Method	Works with instance attributes	☑	✗

Method Type	Description	Uses self ?	Uses cls ?
Class Method	Works with class attributes	✗	☑
Static Method	Does not use class or instance variables	✗	✗

Example

```
class Example:
    class_var = "I am a class variable"

    def instance_method(self):
        return "Instance Method"

    @classmethod
    def class_method(cls):
        return cls.class_var

    @staticmethod
    def static_method():
        return "Static Method"

obj = Example()
print(obj.instance_method()) # Output: Instance Method
print(Example.class_method()) # Output: I am a class variable
print(Example.static_method()) # Output: Static Method
```

Summary of OOP Concepts

Concept	Description	Example
Class	A blueprint for creating objects	<code>class Car:</code>

Concept	Description	Example
Object	An instance of a class	<code>car1 = Car()</code>
Encapsulation	Restrict direct access to data	<code>self.__balance</code>
Inheritance	A class inherits from another class	<code>class Dog(Animal)</code>
Polymorphism	Using the same method in different ways	<code>def fly(self)</code>
Abstraction	Hiding implementation details	<code>@abstractmethod</code>
Magic Methods	Special methods like <code>__str__()</code>	<code>def __len__(self)</code>
Class Methods	Works with class variables	<code>@classmethod</code>
Static Methods	Independent of class and instance	<code>@staticmethod</code>

List Comprehension

List comprehension is a **concise** and **efficient** way to create lists in Python. It allows you to generate lists in a **single line of code**, making your code more readable and Pythonic.

1. Basic Syntax

```
[expression for item in iterable]
```

- `expression` → The operation to perform on each item
- `item` → The variable representing each element in the iterable
- `iterable` → The data structure being iterated over (list, range, etc.)

Example: Creating a list of squares

```
squares = [x**2 for x in range(5)]  
print(squares) # Output: [0, 1, 4, 9, 16]
```

2. Using if Condition in List Comprehension

Example: Filtering even numbers

```
evens = [x for x in range(10) if x % 2 == 0]  
print(evens) # Output: [0, 2, 4, 6, 8]
```

3. Using if-else Condition in List Comprehension

Example: Replacing even numbers with "Even" and odd numbers with "Odd"

```
numbers = ["Even" if x % 2 == 0 else "Odd" for x in range(5)]  
print(numbers) # Output: ['Even', 'Odd', 'Even', 'Odd', 'Even']
```

4. Nested Loops in List Comprehension

Example: Creating pairs from two lists

```
pairs = [(x, y) for x in range(2) for y in range(3)]  
print(pairs) # Output: [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1),
```

5. List Comprehension with Functions

Example: Converting a list of strings to uppercase

```
words = ["hello", "world", "python"]
upper_words = [word.upper() for word in words]
print(upper_words) # Output: ['HELLO', 'WORLD', 'PYTHON']
```

6. List Comprehension with Nested List Comprehension

Example: Flattening a 2D list

```
matrix = [[1, 2], [3, 4], [5, 6]]
flattened = [num for row in matrix for num in row]
print(flattened) # Output: [1, 2, 3, 4, 5, 6]
```

7. List Comprehension with Set and Dictionary Comprehensions

Set Comprehension

```
unique_numbers = {x for x in [1, 2, 2, 3, 4, 4]}
print(unique_numbers) # Output: {1, 2, 3, 4}
```

Dictionary Comprehension

```
squared_dict = {x: x**2 for x in range(5)}
print(squared_dict) # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

8. When to Use List Comprehensions?

- You need to create a list in a **single line**
- The logic is **simple and readable**
- You want to improve **performance** (faster than loops)

Avoid when: - The logic is **too complex** (use a standard loop instead for clarity)

9. Performance Comparison: List Comprehension vs. Loop

```
import time

# Using a for loop
start = time.time()
squares_loop = []
for x in range(10**6):
    squares_loop.append(x**2)
print("Loop time:", time.time() - start)

# Using list comprehension
start = time.time()
squares_comp = [x**2 for x in range(10**6)]
print("List Comprehension time:", time.time() - start)
```

List comprehensions are generally faster than loops because they are optimized internally by Python.

Summary

Concept	Example
Basic List Comprehension	<code>[x**2 for x in range(5)]</code>
With Condition (<code>if</code>)	<code>[x for x in range(10) if x % 2 == 0]</code>
With <code>if-else</code>	<code>["Even" if x % 2 == 0 else "Odd" for x in range(5)]</code>
Nested Loop	<code>[(x, y) for x in range(2) for y in range(3)]</code>
Flatten 2D List	<code>[num for row in matrix for num in row]</code>
Set Comprehension	<code>{x for x in [1, 2, 2, 3]}</code>
Dictionary Comprehension	<code>{x: x**2 for x in range(5)}</code>

Lambda Functions

A **lambda function** in Python is an **anonymous, single-expression function** defined using the `lambda` keyword. It is commonly used for **short, throwaway functions** where a full function definition is unnecessary.

1. Syntax of Lambda Functions

`lambda` arguments: expression

- `lambda` → Keyword to define a lambda function
- arguments → Input parameters (comma-separated)
- expression → The operation performed (must be a **single** expression, not multiple statements)

Example: Simple Lambda Function

```
square = lambda x: x ** 2
print(square(5)) # Output: 25
```

2. Using Lambda Functions with `map()`, `filter()`, and `reduce()`

2.1 Using `map()` with Lambda

Applies a function to each element of an iterable.

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared) # Output: [1, 4, 9, 16]
```

2.2 Using `filter()` with Lambda

Filters elements based on a condition.

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # Output: [2, 4, 6]
```

2.3 Using `reduce()` with Lambda

Reduces an iterable to a single value (requires `functools.reduce`).

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, numbers)
print(product) # Output: 24
```

3. Lambda with Multiple Arguments

Example: Adding Two Numbers

```
add = lambda x, y: x + y
print(add(3, 7)) # Output: 10
```

Example: Finding the Maximum of Two Numbers

```
maximum = lambda x, y: x if x > y else y
print(maximum(10, 5)) # Output: 10
```

4. Lambda in Sorting Functions

Sorting a List of Tuples

```
students = [("Alice", 85), ("Bob", 78), ("Charlie", 92)]
students.sort(key=lambda student: student[1]) # Sort by score
print(students) # Output: [('Bob', 78), ('Alice', 85), ('Charlie', 92)]
```

5. When to Use Lambda Functions?

Use Lambda Functions When:

- The function is short and simple.

- Used **temporarily** inside another function (e.g., `map` , `filter`).
- Avoiding defining a full function with `def` .

Avoid Lambda Functions When:

- The function is **complex** (use `def` for readability).
- Multiple operations/statements are needed.

Summary

Feature	Example
Basic Lambda Function	<code>lambda x: x**2</code>
With <code>map()</code>	<code>map(lambda x: x**2, numbers)</code>
With <code>filter()</code>	<code>filter(lambda x: x % 2 == 0, numbers)</code>
With <code>reduce()</code>	<code>reduce(lambda x, y: x * y, numbers)</code>
Multiple Arguments	<code>lambda x, y: x + y</code>
Sorting with Lambda	<code>sort(key=lambda x: x[1])</code>